2026-2학기 AI와 공학도구를 활용한 수학수업

10주차 (2026.11.5.)

Streamlit 기초

Su Bhin Hwang

한양대학교 사범대학 수학교육과

1차시: Streamlit과 배포의 큰 그림

2차시: 기본 함수 실습, 첫 앱 배포

Streamlit 이해하기

Streamlit 왜 필요한가요?

Python & Streamlit, 오늘 강의의 목표

중복된 도서기록 찾기!
이걸 종이로 인쇄해서
하나하나 비교해야 해?



코드는 짰다!
🎯 근데… 매번 파이썬 켜야
해? □ □ □ □ □ □ □ ?

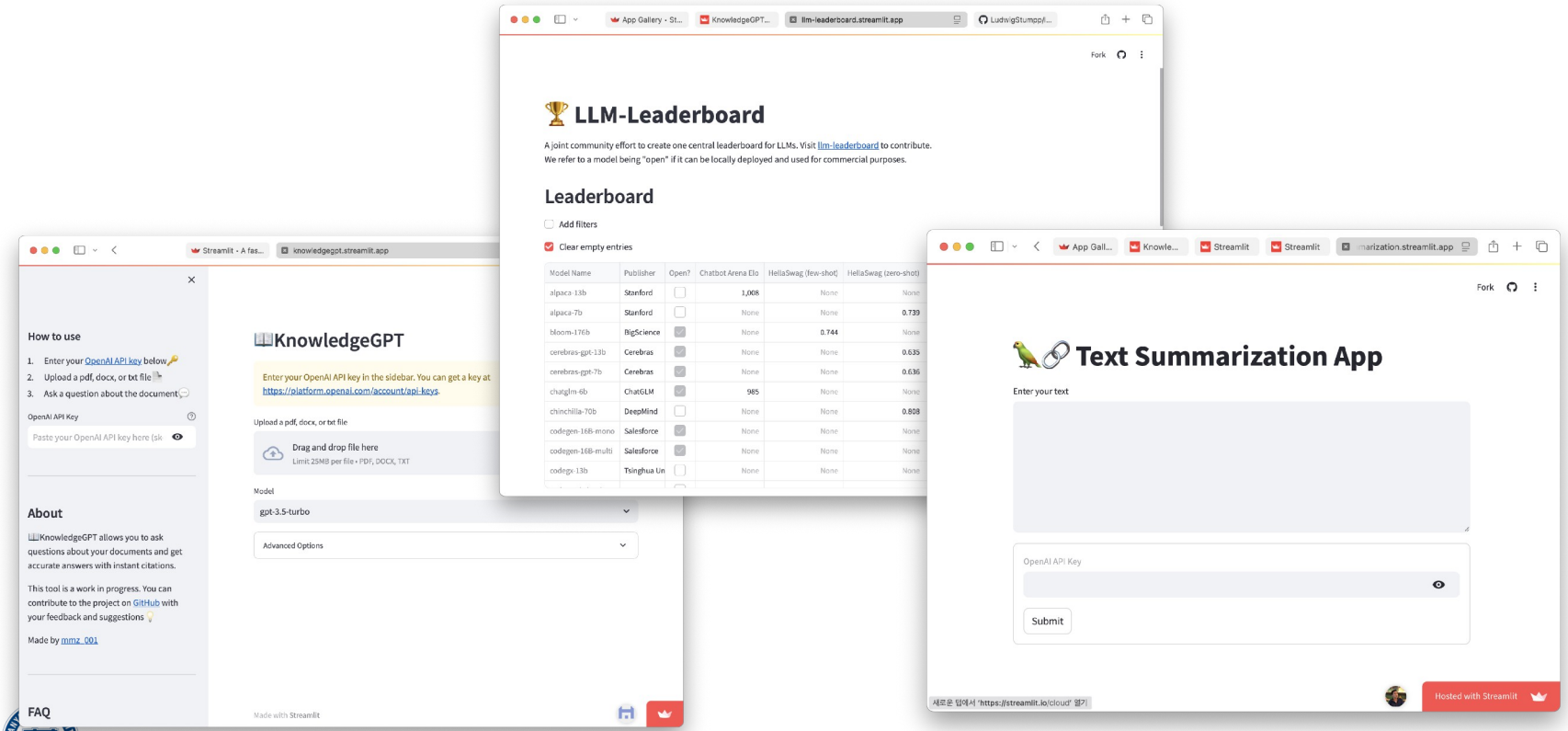


웹앱으로 변신 완료! 🚀
핸드폰에서도 바로 쓰고,
친구한테도 공유하기!



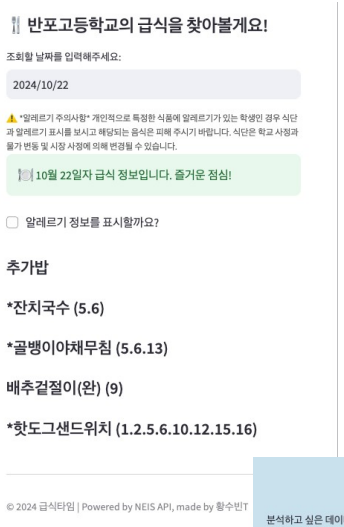
Streamlit으로 제작한 LLM 앱 예시

바로가기 : https://streamlit.io/gallery

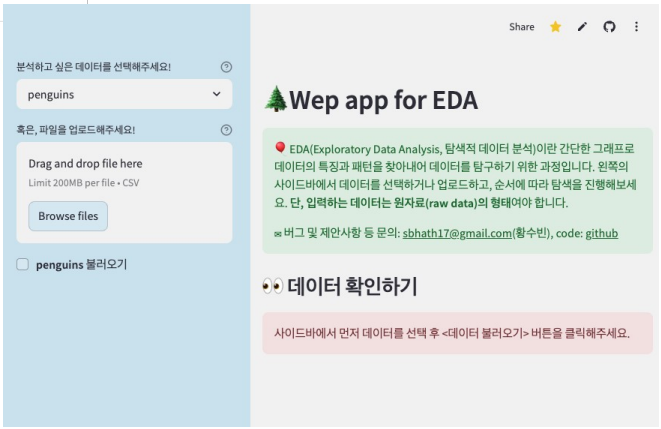


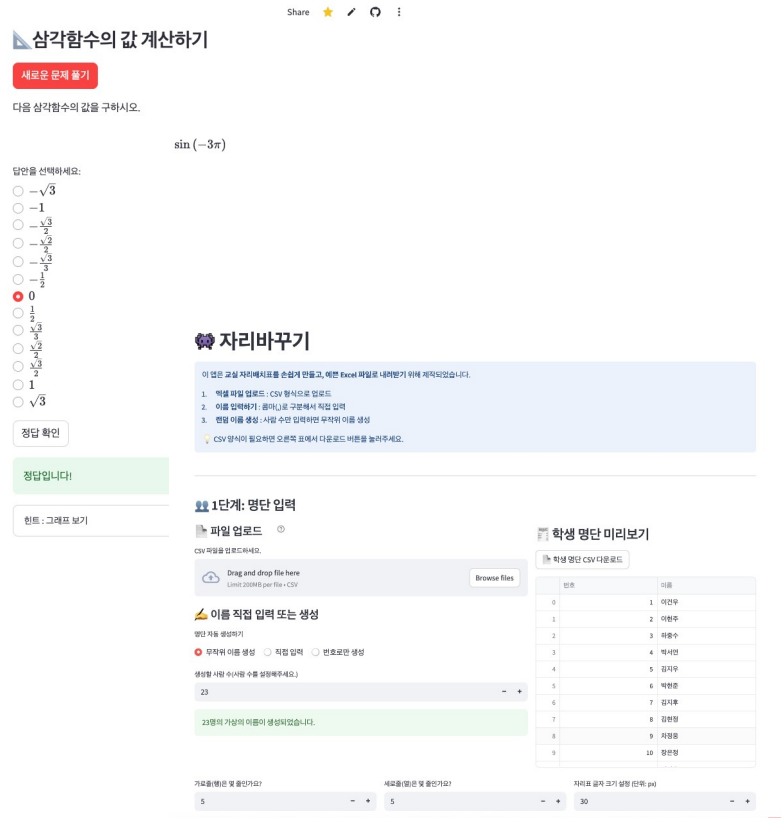
Streamlit으로 만든 앱 예시

바로가기 : https://share.streamlit.io/user/surihub









Streamlit이 그래서 뭔가요?

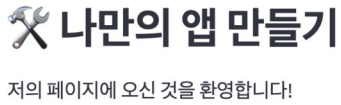
- 단 몇 줄의 **python** 코드로 나만의 앱을 만들 수 있는 **python** 라이브러리

pages > ex.py > ...

```python
import streamlit as st

st.title("🛠️ 나만의 앱 만들기")
st.write("저의 페이지에 오신 것을 환영합니다!")
```





Notion으로도 충분하지 않나요?

- 단 몇 줄의 **python** 코드로 나만의 '**interactive**'한 앱을 만들 수 있는 라이브러리

pages > ex.py > ...

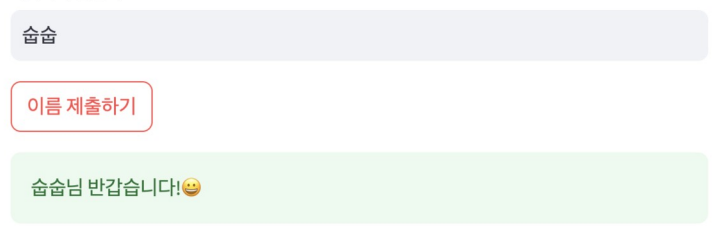
```python
import streamlit as st

st.title("🛠️ 나만의 앱 만들기")
st.write("저의 페이지에 오신 것을 환영합니다!")


name = st.text_input("이름이 무엇인가요?")
if st.button("이름 제출하기"):

    st.success(f"{name}님 반갑습니다!😀")

```



Google Sites로도 충분하지 않나요?

- 단 몇 줄의 **python** 코드**만으로** 나만의 **AI 기반**의 ‘interactive’한 앱을 만들 수 있는 라이브러리

pages > ex.py > ...

```python
import streamlit as st

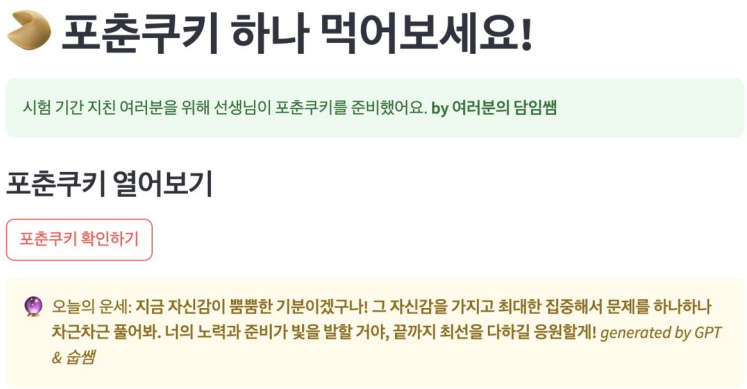
st.title("🛠️ 나만의 앱 만들기")
st.write("저의 페이지에 오신 것을 환영합니다!")


name = st.text_input("이름이 무엇인가요?")
if st.button("이름 제출하기"):

    st.success(f"{name}님 반갑습니다!😀")

```





Streamlit Playground로 연습하기

- streamlit.io/playground: 설치 없이 브라우저에서 바로 쓰는 Streamlit (기능 제한판)
- 왼쪽에서 코드를 고치면 오른쪽 앱에 바로 반영
- 예제: Blank, Hello, Charts, Dataframes, LLM chat, Computer vision, Geospatial
- 연습
  - Hello 예제의 제목, 문구, 위젯 바꿔 보기
  - Blank에서 이름 입력과 버튼이 있는 앱 만들기 (앞의 코드 예시)
  - Charts, Dataframes 예제로 표와 그래프가 그려지는 코드 확인
- 설치, 저장소, 배포가 필요한 앱 제작은 Codespaces와 Streamlit Cloud에서

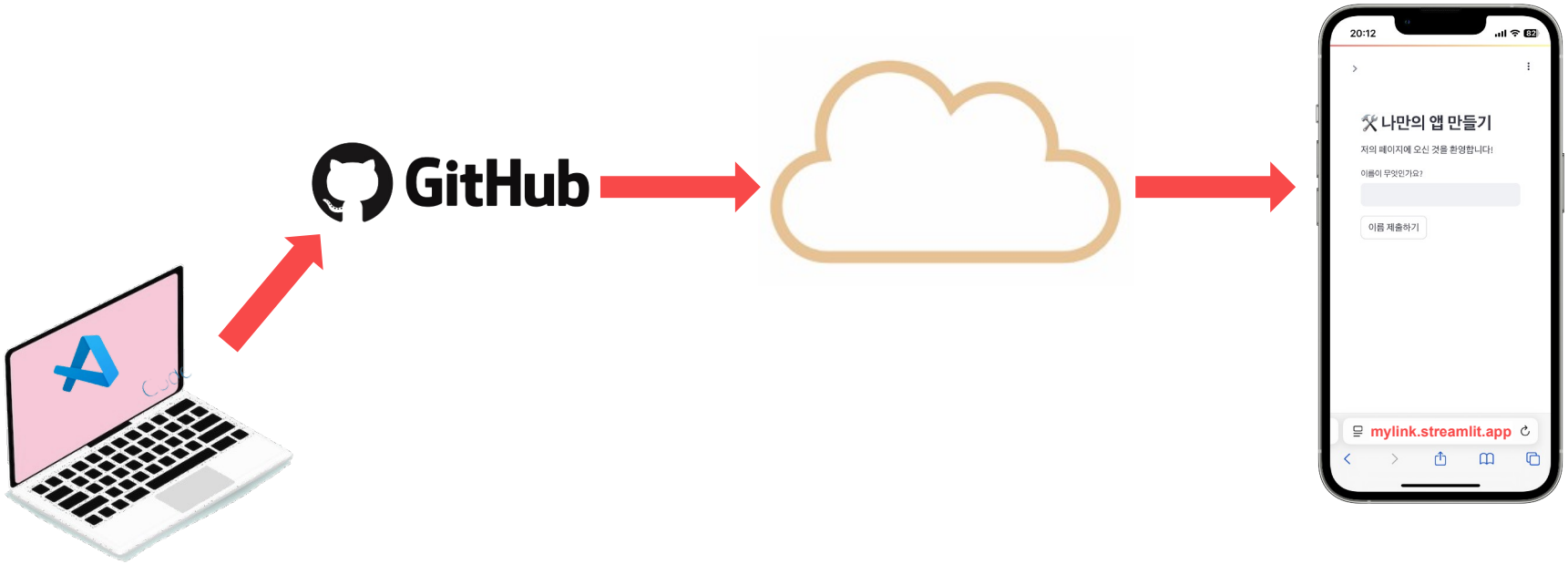
보통 웹앱을 배포하는 방법

- Html, css, JavaScript, Flask, Django, 서버비용…



Streamlit으로 배포하기 도식 1️⃣

- 로컬 환경에 Visual Studio Code를 설치, git 설치, git commit을 한 후 streamlit cloud에 배포



Streamlit으로 배포하기 도식 2️⃣

- 웹(github codespaces)에서 코드를 입력하고 저장(commit)하여 바로 streamlit cloud에 배포!



즉 브라우저에서 코드를 작성하고, 바로 배포한다!

For

지금까지의 도구와 Streamlit

지금까지 사용한 도구

Amplify, Desmos, GeoGebra, CODAP

만들어진 도구의 기능 안에서 수업 설계
도구에 없는 기능은 구현하기 어려움

Streamlit

Python + AI 코딩 도구

수업에 필요한 앱을 직접 제작
필요한 기능을 정하고 구현

과제1에서 메모한 '이 도구에서 불가능했던 것' → **과제3, 프로젝트에서 직접 해결**

앱 만들기에 쓰는 도구들

코드 만들기

Python
코드를 쓰는 언어

Streamlit
Python으로 웹앱 화면을 만드는 라이브러리

Codespaces
브라우저에서 쓰는 VS Code, 설치 없이 코드 작성과 실행

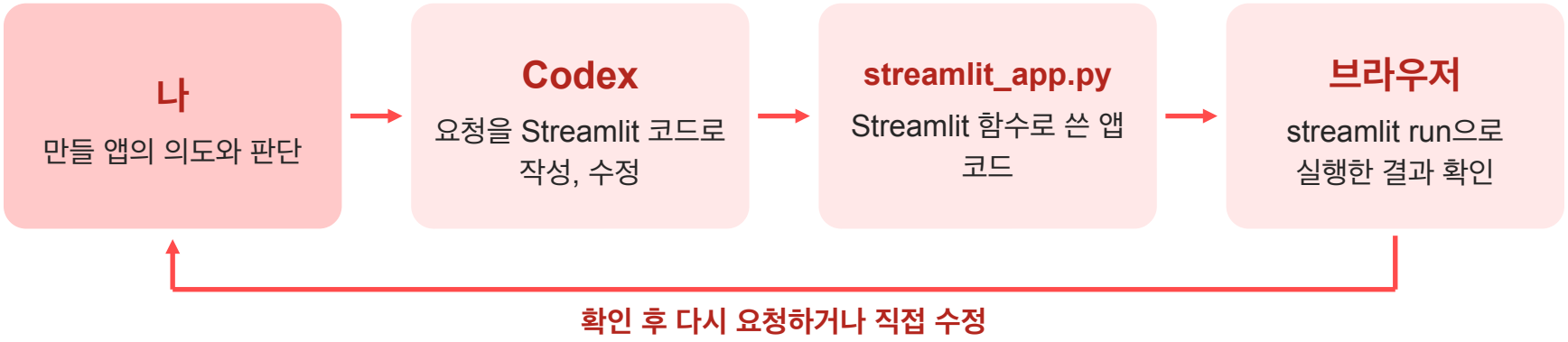
Codex
요청을 받아 코드를 쓰고 고치는 AI 코딩 에이전트

인터넷에 공개하기

GitHub
코드를 올려 두고 수정 이력을 관리하는 저장소

Streamlit Community Cloud
GitHub의 코드를 받아 앱을 인터넷에 띄우는 무료 서버

Codex와 Streamlit의 관계



- **Streamlit**은 앱을 만드는 재료, **Codex**는 그 재료로 코드를 써 주는 도구
- Codex 없이도 Streamlit 앱 제작 가능, ChatGPT나 Claude로 대신 가능
- 실행 결과가 맞는지 판단하는 것은 나, 문제를 이해해야 검증 가능

배포(Deployment)란?

배포: 내가 만든 앱을 다른 사람도 쓸 수 있게 인터넷에 올리는 것

Codespaces에서 실행

streamlit run으로 작업 공간 안에서 실행

주소: **미리보기 주소 (포트 8501)**

작업하는 나만 확인

Codespaces를 끄면 앱도 종료

배포 →

배포된 앱

인터넷 서버에서 실행

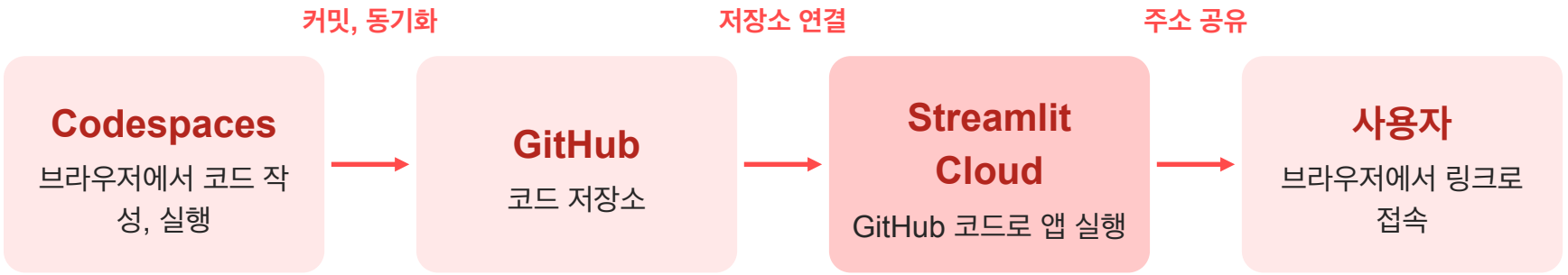
주소: ○○○**.streamlit.app**

링크만 있으면 누구나 접속, 휴대폰 포함

Codespaces를 꺼도 유지

보통의 배포: 서버, 도메인, 비용 필요 → **Streamlit: GitHub 연결만으로 무료 배포**

배포 흐름



- **커밋(commit)**: 소스 제어 탭에서 변경 내용 저장, 무엇을 왜 바꿨는지 메모
- **동기화(sync)**: 푸시(GitHub에 업로드)와 풀(GitHub에서 가져오기)로 최신 상태 맞추기
- 커밋, 동기화하면 배포된 앱에 자동 반영
- 배포 후 수정: 앱 화면의 수정 버튼으로 Codespaces 열기, 로그인했을 때만 보임

Streamlit 실습하기

첫 앱 만들고 배포하기

Streamlit 앱 폴더 구조

```
my-app/
    streamlit_app.py
    requirements.txt
    .gitignore
    pages/
        page1.py
    .streamlit/
        secrets.toml
```

- **streamlit_app.py**: 앱 본체, 화면과 기능
- **requirements.txt**: 앱에 필요한 패키지 목록, 배포 서버가 읽고 설치
- **pages/**: 여러 페이지로 된 앱의 페이지별 파일
- **.streamlit/secrets.toml**: API 키 같은 비밀값 보관
- **.gitignore**: GitHub에 올리지 않을 파일 목록

Streamlit 기본 함수 한눈에 보기

화면 꾸미기	st.set_page_config, st.title, st.write, st.markdown, st.image
입력받기	st.text_input, st.number_input, st.slider, st.selectbox, st.radio
버튼과 조건문	st.button + if 문
외부 도구 넣기	iframe으로 GeoGebra, Desmos, CODAP 등 임베드
AI 연결	OpenAI API: model, messages(system, user), temperature

바퀴를 재발명하지 않기: 이미 있는 도구와 기능은 가져와 조합

실습 흐름

- GitHub 가입, GitHub 계정으로 Streamlit Community Cloud 가입
- Streamlit에서 앱 만들기 (Blank app 또는 예제 fork)
    - GitHub 저장소와 앱 주소가 함께 생성, GitHub Codespaces로 열기
- Codespaces에서 **streamlit_app.py** 수정, 기본 함수를 바꾸며 화면 변화 확인
- Codex에 요청해 기능 추가, 코드를 읽고 직접 고쳐 보기
- 소스 제어 탭에서 커밋, 동기화 후 배포된 앱에서 확인
- 배포 링크 공유, 서로의 앱 접속

배포할 때 자주 생기는 문제

- **모듈 설치 오류**
  - pip install로 설치한 패키지는 requirements.txt에도 추가
- **API 키 관리**
  - secrets.toml은 .gitignore에 넣어 GitHub 업로드 제외
  - 배포한 앱에서는 Streamlit Cloud 앱 설정의 Secrets에 입력
- **배포 오류 확인**
  - Manage app에서 로그 확인, 에러 메시지를 Codex에 붙여 넣어 원인 찾기
- **수정이 반영되지 않거나 앱이 잠든 경우**
  - Reboot app, ‘Yes, get this app back up!’ 클릭
  - Codespaces도 오래 켜 두면 중지됨, 다시 열어 재시작

FAQ

- Local URL (http://localhost:8502)
  - **접속 기기**: 앱을 직접 실행하고 있는 본인 컴퓨터
  - **특징**: 내 컴퓨터 자신만을 가리키는 루프백(Loopback) 주소입니다. 다른 스마트폰이나 타인의 PC에서는 이 주소로 들어올 수 없습니다.
- Network URL (http://10.64.253.227:8502)
  - **접속 기기**: 같은 와이파이나 내부망에 연결된 스마트폰, 태블릿, 동료의 PC 등
  - **특징**: 동일한 네트워크 범위 내에 있는 다른 기기에서 내 작업 결과물을 미리 테스트하거나 보여줄 때 사용합니다. (단, 같은 와이파이 망을 벗어난 외부 인터넷 영역에서는 접속되지 않습니다.)

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8502
Network URL: http://10.64.253.227:8502
```

앱을 만들며 고민할 점

- Streamlit, Google Sites, Notion, Amplify, CODAP, 누구에게나 만능인 도구는 없음
- Streamlit과 Python의 마스터 (X)
- **도구와 환경에 제한되지 않는 나만의 앱 (O)**
- 앱을 만드는 것 자체는 더 이상 어렵지 않음
    - **이 앱이 왜 필요한지, 수업의 어디에 쓰는지**가 핵심
- 프로젝트: 기존 도구로 불가능했던 장면을 직접 만든 앱으로 해결